

P3628R0

`break label;` and `continue label;`, `tl;` `dr`

Published Proposal, 2025-02-12

This version:

<https://eisenwave.github.io/cpp-proposals/break-continue-label-tldr.html>

Author:

[Jan Schultke](#)

Audience:

SG22, SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](#)

Abstract

Abridged version of [\[P3568R0\]](#).

Table of Contents

- 1 Introduction**
- 2 Motivation**
 - 2.1 No good alternative
 - 2.1.1 `goto`
 - 2.1.2 Immediately invoked lambda expression (IILE)
 - 2.1.3 Mutable `bool` state
 - 2.2 Argumentum ad populum

3 Design Considerations

- 3.1 Alternative `break` and `continue` forms
- 3.2 `constexpr break label;` and `continue label;`
- 3.3 Syntax warzone
 - 3.3.1 N3377 benefits
 - 3.3.2 N3377 problems
 - 3.3.2.1 Breaking precedent of most prior art
 - 3.3.2.2 Repetition
 - 3.3.2.3 Extendability
 - 3.3.2.4 Blocking contextual keywords
 - 3.3.2.5 Labeling loops expanded from macros
 - 3.4 Changes to labels
 - 3.4.1 New labels - `goto`
 - 3.4.2 New labels - nesting?
 - 3.4.3 New labels - direct duplicates?
 - 3.4.4 New labels - what about `break label` for loops with more than one label?

4 Impact on existing code

5 Implementation experience

6 Proposed wording

References

Normative References

Informative References

§ 1. Introduction

Proposed functionality:

```
outer: for (auto x : xs) {  
    for (auto y : ys) {  
        if (/* ... */) {  
            continue outer; // OK, continue applies to outer for loop  
            break outer;    // OK, break applies to outer for loop  
        }  
    }  
}
```

```

    }
  }
}

switch_label: switch (/* ... */) {
  default: while (true) {
    if (/* ... */) {
      break switch_label; // OK, break applies to switch, not to whi
    }
  }
}

break outer;      // error: cannot break loop from the outside
goto outer;       // OK, used to be OK, and is unaffected by this proposa

switch_label;;    // OK, labels can be reused
goto switch_label; // error: jump target is ambiguous

```

- Proposed syntax (except label relaxations) is identical to [\[N3355\]](#), accepted into C2y.

Decisions required:

1. Do we want `break label` at all?

2. Do we want this syntax?

Functionality was proposed before in [\[N3879\]](#) and rejected at Rapperswil 2014 ([\[N4327\]](#)):

Straw poll, proposal as a whole:

SF	F	N	A	SA
1	1	1	13	10

"break label;" + "continue label;"

SF	F	N	A	SA
3	8	4	9	3

However, things have changed since 2014:

- C2y acceptance.
- `constexpr` much more common now, but `goto` workaround doesn't work.

- `break label` staple of new languages; see Rust, Kotlin, Cpp2.

§ 2. Motivation

- `break label` and `continue` label useful for controlling nested loops/switches.
- Early `return` not always viable alternative:
 - not viable in every function
 - MISRA-C++:2008 Rule 6-6-5 "A function shall have a single point of exit at the end of the function"
 - does not replace `continue`

§ 2.1. No good alternative

```
process_files: for (const File& text_file : files) {  
    for (std::string_view line : text_file.lines()) {  
        if (makes_me_angry(line)) {  
            continue process_files;  
        }  
        consume(line);  
    }  
    std::println("Processed {}", text_file.path());  
}  
std::println("Processed all files");
```

- `continue process_files` works really well, very readable.
- `break;` would skip `std::println`.

§ 2.1.1. `goto`

```
for (const File& text_file : files) {  
    for (std::string_view line : text_file.lines()) {  
        if (makes_me_angry(line)) {  
            goto done_with_file;  
        }  
        consume(line);  
    }  
}
```

```

    }
    std::println("Processed {}", text_file.path());
    done_with_file:
}
std::println("Processed all files");

```

Works, but:

- `goto` cannot cross (non-vacuous) initialization (problematic if we add variable before `std::println`)
- not `constexpr`
- Discouraged heavily (CppCoreGuidelines, MISRA, etc.)
- No community consensus when *exactly* `goto` is OK.
- Meaning of `goto` unclear until seeing label location.
- Bad label name = confusing control flow.
- Super controversial.

§ 2.1.2. Immediately invoked lambda expression (IILE)

```

for (const File& text_file : files) {
    [&] {
        for (std::string_view line : text_file.lines()) {
            if (makes_me_angry(line)) {
                return;
            }
            consume(line);
        }
        std::println("Processed {}", text_file.path());
    }();
}
std::println("Processed all files");

```

Works, but:

- Can no longer `break` outer loop.
- More indentation level.
- Cost in constant evaluation, debug builds.

- `return;` expresses intent poorly.

§ 2.1.3. Mutable `bool` state

```
for (const File& text_file : files) {
    bool success = true;
    for (std::string_view line : text_file.lines()) {
        if (makes_me_angry(line)) {
            success = false;
            break;
        }
        consume(line);
    }
    if (success) {
        std::println("Processed {}", text_file.path());
    }
}
std::println("Processed all files");
```

Works, but:

- More mutable state to keep track of.
- Way longer, extra `if`.
- Needs one `bool` per loop, doesn't scale.

§ 2.2. Argumentum ad populum

- `break label` and `continue label` very popular in other languages.
- This proposal: positive reception on Reddit.
- See poll below.

Should C++ have "break label" and "continue label" statements to apply break/continue to nested loops or switches?

SF	F	N	A	SA
21	21	12	6	4

Source: Together C & C++ (Discord server).

Language	Syntax	Labeled <code>break</code> s	Labeled <code>continue</code> s	Σ <code>break</code> <code>continue</code>	<code>goto</code> s
Java	<code>label: for</code> <code>(...)</code> <code>break label;</code> <code>continue label;</code>	424K files	152K files	576K files	N/A
JavaScript	<code>label: for</code> <code>(...)</code> <code>break label;</code> <code>continue label;</code>	53.8K files	68.7K files	122.5K files	N/A
Perl	<code>label: for</code> <code>(...)</code> <code>last label;</code> <code>next label;</code>	34.9K files	31.7K files	66.6K files	16.9K files
Rust	<code>label: for</code> <code>(...)</code> <code>break 'label;</code> <code>continue</code> <code>'label;</code>	30.6K files	29.1K files	59.7K files	N/A
TypeScript	<code>label: for</code> <code>(...)</code> <code>break label;</code> <code>continue label;</code>	11.6K files	9K files	20.6K files	N/A
Swift	<code>label: for ...</code> <code>break label</code> <code>continue label</code>	12.6K files	5.6K files	18.2K files	N/A
Kotlin	<code>label@ for</code> <code>(...)</code> <code>break@label</code> <code>continue@label</code>	8.7K files	7.6K files	16.3K files	N/A
D	<code>label: for</code> <code>(...)</code> <code>break label;</code> <code>continue label;</code>	3.5K files	2.6K files	6.1K files	12.3K files
Go	<code>label: for ...</code> <code>break label;</code> <code>continue label;</code>	270 files	252 files	522	1.5K files

Cpp2 (cppfront)	<code>label: for ...</code> <code>break label;</code> <code>continue label;</code>	N/A	N/A	N/A	N/A
C	<code>label: for</code> <code>(...)</code> <code>break label;</code> <code>continue label;</code>	N/A	N/A	N/A	7.8M files

§ 3. Design Considerations

§ 3.1. Alternative `break` and `continue` forms

- `break N` (e.g. `break 2;`)
- `break while` (i.e. use keyword as label)
- `break statement` (e.g. `break break;`)

No, thanks! Received poorly, too exotic. WG14 wants `break label;`.

§ 3.2. `constexpr break label;` and `continue label;`

Yes, absolutely! See motivation; easier to implement than `constexpr goto` because almost exists already:

```
while (/* ... */) {
    if (/* ... */) {
        { { { { { break; } } } } }
    }
}
```

- Arbitrarily deeply nested `break` already implemented.
- `break label` just skips some statements and breaks more, easy.

§ 3.3. Syntax warzone

- I like [\[N3355\]](#), accepted into C2y.

- Competing proposal [\[N3377\]](#) seen at Graz, this month.

[\[N3377\]](#) syntax:

```
for outer (/* ... */) {  
    while (/* ... */) break outer;  
    while outer (/* ... */) {  
        // OK, applies to the enclosing while loop  
    }  
}
```

§ 3.3.1. N3377 benefits

- *block-names* don't have to be unique, and this is good for macros.
 - Counterpoint: I relax labels instead, no need for [\[N3377\]](#) syntax.
- Disambiguation between `goto` targets and `break` targets.
 - Counterpoint: Impact may be overstated, could disambiguate with naming convention, linter scripts.

NOTE: There are some other minor arguments for [\[N3377\]](#), but we don't have the time.

§ 3.3.2. N3377 problems

§ 3.3.2.1. *Breaking precedent of most prior art*

Breaks most prior art; see language stats above and examples below:

Perl:

```
goto LINE;  
LINE: while (true) {  
    last LINE; # like our proposed break LINE  
}
```

Go:

```
goto OuterLoop
OuterLoop: for {
    break OuterLoop
}
```

D:

```
goto outer;
outer: while (true) {
    break outer;
}
```

However, **Ada:**

```
goto Target;
<<Target>>
Outer: loop
    exit Outer; -- like our proposed break Outer
end loop Outer;
```

- `break label` and `continue label` has decades of precedent in other languages.
- C++ often used in conjunction with other languages, ergo common syntax = good.

§ 3.3.2.2. Repetition

[N3377] is bad when loop is targeted by `break` and `goto`:

```
goto outer;
// ...
outer: while outer(true) {
    while(true) {
        break outer;
    }
}
```

- Option A: copy/paste name, but DRY!
- Option B: two different names for same loop?!

§ 3.3.2.3. *Extendability*

```
label: {  
    // OK in Java, JS, TS  
    break label;  
}
```

- Not proposed, but who knows next 40 years?
- [\[N3377\]](#) syntax generally harder to apply to new features, more existing ones.

§ 3.3.2.4. *Blocking contextual keywords*

```
while parallel(/* ... */)
```

- `parallel` cannot be contextual keyword if taken by user.
- Precedent: `if constexpr`.
- To be fair, [\[N3377\]](#) floats idea `while :parallel: ()`.
 - But then why not put label in front?

§ 3.3.2.5. *Labeling loops expanded from macros*

Example from uthash:

```
#define HASH_ITER(hh, head, el, tmp) for (/* ... */)
```

With [\[N3355\]](#):

```
struct my_struct *current_user, *tmp;

outer: HASH_ITER(hh, users, current_user, tmp) {
    for (/* ... */) {
        if (/* ... */) break outer;
    }
}
```

With [\[N3377\]](#): ask library author to change macro or make your own.

§ 3.4. Changes to labels

As stated before, we relax labels as follows:

```
outer: while (true) {
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

outer: while (true) { // OK, reusing label is permitted
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

goto outer; // error: ambiguous jump target
```

- OK in Java and JavaScript.
- With `'outer` and `'inner` instead, this code is OK Rust.

§ 3.4.1. New labels - `goto`

```
x: f(); // OK
x: g(); // OK
```

```
goto x; // error: jump is ambiguous
```

No existing code broken.

§ 3.4.2. New labels - nesting?

Another case to consider is the following:

```
l: while (true) {  
    l: for (true) { // OK  
        break l; // breaks for loop  
    }  
}
```

- OK in Rust 'l', but not in JS, Java.
- Motivation: macros.

§ 3.4.3. New labels - direct duplicates?

```
l: l: l: l: for(); // OK
```

- Motivation:
 - Why not?
 - Macros.

§ 3.4.4. New labels - what about `break label` for loops with more than one label?

```
x: y: while (true) {  
    break x; // OK  
}
```

- Motivation: OK in C2y ([\[N3355\]](#) and [\[N3377\]\(?\)](#)).

§ 4. Impact on existing code

None.

§ 5. Implementation experience

- Tons of other languages have this; it works.
- A GCC implementation of [\[N3355\]](#)
- An LLVM implementation is W.I.P.

§ 6. Proposed wording

The wording is relative to [\[N5001\]](#).

Update [\[stmt.label\]](#) paragraph 1 as follows:

A label can be added to a statement or used anywhere in a *compound-statement*.

label:

*attribute-specifier-seq*_{opt} *identifier* :

*attribute-specifier-seq*_{opt} **case** *constant-expression* :

*attribute-specifier-seq*_{opt} **default** :

labeled-statement:

label statement

The optional *attribute-specifier-seq* appertains to the label. ~~The only use of a label with an *identifier* is as the target of a **goto**. No two labels in a function shall have the same identifier.~~ A label can be used in a **goto** statement ([\[stmt.goto\]](#)) before its introduction.

[Note: Multiple identical labels within the same function are permitted, but such duplicate labels cannot be used in a **goto** statement. — end note]

In [\[stmt.label\]](#) insert a new paragraph after paragraph 1:

A label **L** of the form *attribute-specifier-seq_{opt} identifier : labels a statement S* if

- **L** is the *label* and **S** is the *statement* of a *labeled-statement X*, or
- **L** labels **X** (recursively).

[Example:

```
a: b: while (0) { }           // both a: and b: label the loop
c: { d: switch (0) {         // unlike c:, d: labels the switch statement
    default: while (0) { }   // default: labels nothing
} }
```

— end example]

NOTE: This defines the term *(to) label*, which is used extensively below. We also don't want **case** or **default** labels to label statements, since this would inadvertently permit **break i** given **case i:**, considering how we word [stmt.break].

Update [stmt.label] paragraph 3 as follows:

A *control-flow-limited statement* is a statement **S** for which:

- a **case** or **default** label appearing within **S** shall be associated with a **switch** statement ([stmt.switch]) within **S**, and
- a label declared in **S** shall only be referred to by a statement (~~[stmt.goto]~~) in **S**.

NOTE: While the restriction still primarily applies to **goto** (preventing the user from e.g. jumping into an **if constexpr** statement), if other statements can also refer to labels, it is misleading to say "statement ([stmt.goto])" as if **goto** was the only relevant statement.



Update [stmt.jump.general] paragraph 1 as follows:

Jump statements unconditionally transfer control.

jump-statement:

```
goto identifier ;  
break identifieropt ;  
continue identifieropt ;  
return expr-or-braced-init-listopt ;  
goto identifier ;
```

NOTE: `goto` is being relocated to the top so that all the jump statements with an *identifier* are grouped together. Of these three, `goto` is being listed first because it models the concept of "jumping somewhere" most literally; every following statement is more sophisticated or even defined as equivalent to `goto` (in the case of `continue`).



Update [\[stmt.break\]](#) paragraph 1 as follows:

A breakable statement is an iteration-statement ([stmt.iter]) or a switch statement ([stmt.switch]). A break statement shall be enclosed by ([stmt.pre]) a breakable statement ~~an iteration-statement~~ ([stmt.iter]) or a ~~switch statement~~ ([stmt.switch]) . If present, the identifier shall be part of a label L which labels ([stmt.label]) an enclosing breakable statement. The break statement causes termination of : ~~the smallest such enclosing statement;~~

- if an identifier is present, the smallest enclosing breakable statement labeled by L,
- otherwise, the smallest enclosing breakable statement.

~~control~~ Control passes to the statement following the terminated statement, if any.

[Example:

```
a: b: while (/* ... */) {
    a: a: c: for (/* ... */) {
        break;                // OK, terminates enclosing for loop
        break a;              // OK, same
        break b;              // OK, terminates enclosing while loop
        y: { break y; }       // error: break does not refer to a breakable
    }
    break c;                  // error: break does not refer to an enclosing
}
break;                       // error: break must be enclosed by a breakable
```

— end example]



Update [stmt.cont] paragraph 1 as follows:

A `continue` statement shall be enclosed by ([`stmt.pre`]) an *iteration-statement* ([`stmt.iter`]). If present, the *identifier* shall be part of a label **L** which labels ([`stmt.label`]) an enclosing *iteration-statement*. The `continue` statement causes control to pass to the loop-continuation portion of ~~the smallest such enclosing statement, that is, to the end of the loop.~~

- if an *identifier* is present, the smallest enclosing *iteration-statement* labeled by **L**,
- otherwise, the smallest enclosing *iteration-statement*.

More precisely, in each of the statements

```
label: while (foo) {  
    {  
        // ...  
    }  
contin: ;  
}
```

```
label: do {  
    {  
        // ...  
    }  
contin: ;  
} while (foo);
```

```
label: for (;;) {  
    {  
        // ...  
    }  
contin: ;  
}
```

~~a `continue` not contained in an an enclosed iteration statement is equivalent to `goto contin`.~~
the following are equivalent to `goto contin`:

- A `continue` not contained in an an enclosed iteration statement.
- A `continue label` not contained in an enclosed iteration statement labeled `label`..



Update [[stmt.goto](#)] paragraph 1 as follows:

The `goto` statement unconditionally transfers control to ~~the~~ a statement labeled (`[stmt.label]`) by the identifier a label in the current function containing *identifier*, but not to a `case` label . The identifier shall be a label located in the current function. There shall be exactly one such label.

§ References

§ Normative References

[N5001]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 17 December 2024. URL: <https://wg21.link/n5001>

§ Informative References

[N3355]

Alex Celeste. *N3355: Named loops, v3*. 2024-09-18. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3355.htm>

[N3377]

Erich Keane. *N3377: Named Loops Should Name Their Loops: An Improved Syntax For N3355*. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3377.pdf>

[N3879]

Andrew Tomazos. *Explicit Flow Control: break label, goto case and explicit switch*. 16 January 2014. URL: <https://wg21.link/n3879>

[N4327]

Ville Voutilanen. *C++ Standard Evolution Closed Issues List (Revision R10)*. 21 November 2014. URL: <https://wg21.link/n4327>

[P3568R0]

Jan Schultke, Sarah Quinones. *break label; and continue label;*. 12 January 2025. URL: <https://wg21.link/p3568r0>